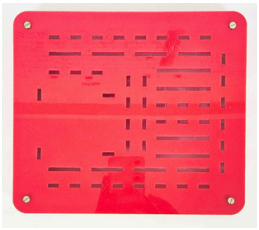
SCIENCEUTSAV®
Science is awesome

I'm a ROBO SCIENTIST

# Laser Security and Smart Detection Systems

## Session 9

# STRUCTURE AND FASTENERS OF MICROCONTROLLERS

1 

2 

3 

4 

5 

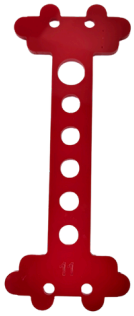
6 

7 

8 

9 

10 

11 

1 

Spacers

2 

L Clamp

3 

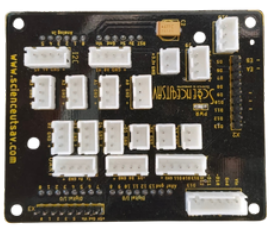
Spacer Screws

4 

Nuts

5 

Screw

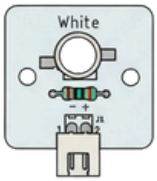
6 

Arduino With Shield

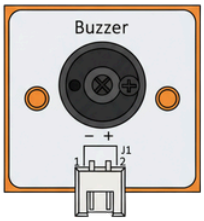
7 

Arduino cable

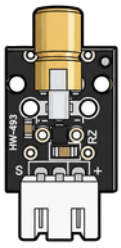
# ELECTRONIC COMPONENTS



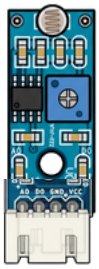

**1-watt LED**
A 1-watt LED lamp is an energy-saving light that uses less electricity. It gives bright light while producing very little heat. It is commonly used in small lamps, indicators, and decorative lighting.




**BUZZER PCB**
A buzzer is an electronic component that generates sound when electric current flows through it. It is commonly utilized in alarms, alerts, and notification systems to provide auditory signals.



**Laser diode**
A Laser diode is a semiconductor device that produces a focused beam of light. It works by converting electrical energy into laser light. Laser diodes are used in barcode scanners, fiber-optic communication, and CD/DVD players.



**LDR sensor**
An LDR sensor is a sensor that detects the intensity of light. Its resistance changes according to the amount of light falling on it. It is used in automatic street lights, alarms, and light-controlled devices.



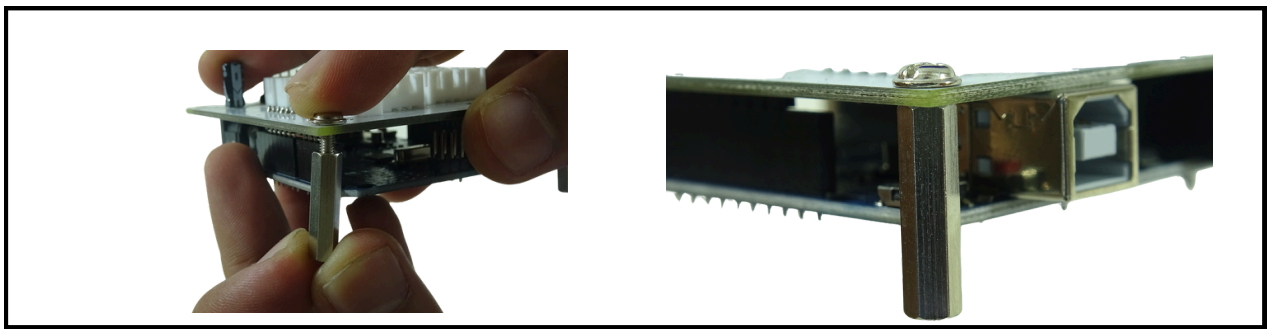
**Push Switch**
A push switch is a button-style switch that operates only when pressed.
It provides temporary input in circuits, such as starting a device or triggering an action.



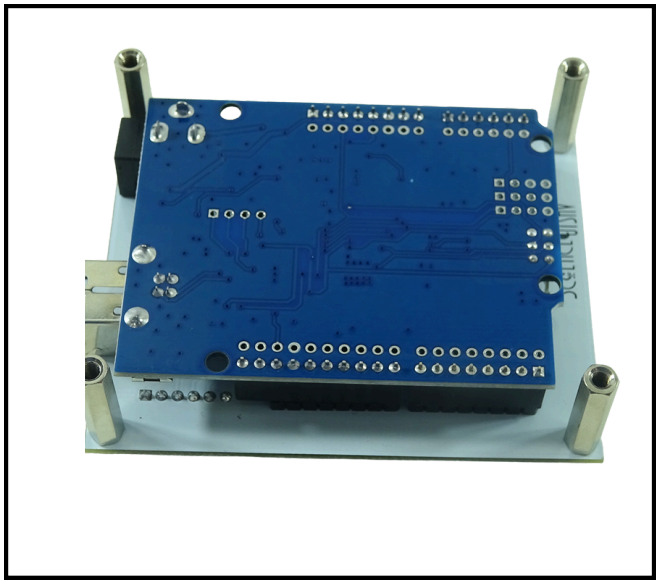

**7-segment display**
7 seg display electronic display device used to show numbers. It has seven LED segments arranged in the shape of the digit 8. By turning ON and OFF different segments, it can display numbers from 0 to 9. It is commonly used in digital clocks, calculators, counters, meters, and electronic device

# Procedure :

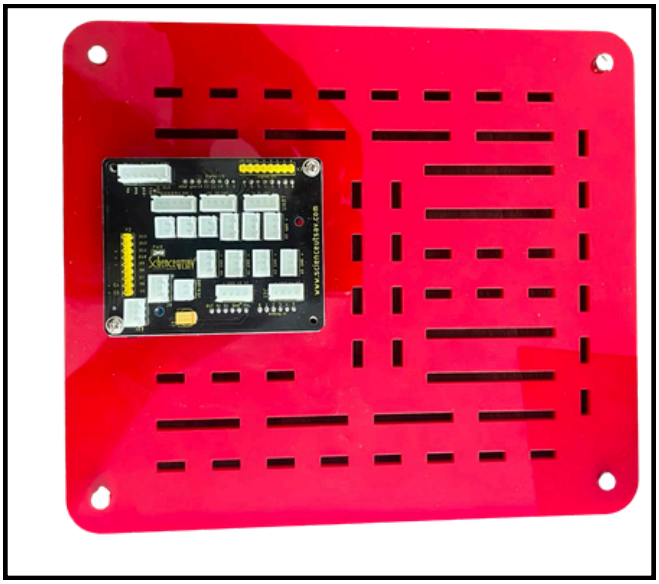
**Step 1 :** Fit the spacer to Arduino Shield as shown below using Spacer screw



**Step 2 :** Repeat the step 1 for all 4 corners of Arduino Shield



**Step 3 :** Place the Arduino Shield to the base board as shown below

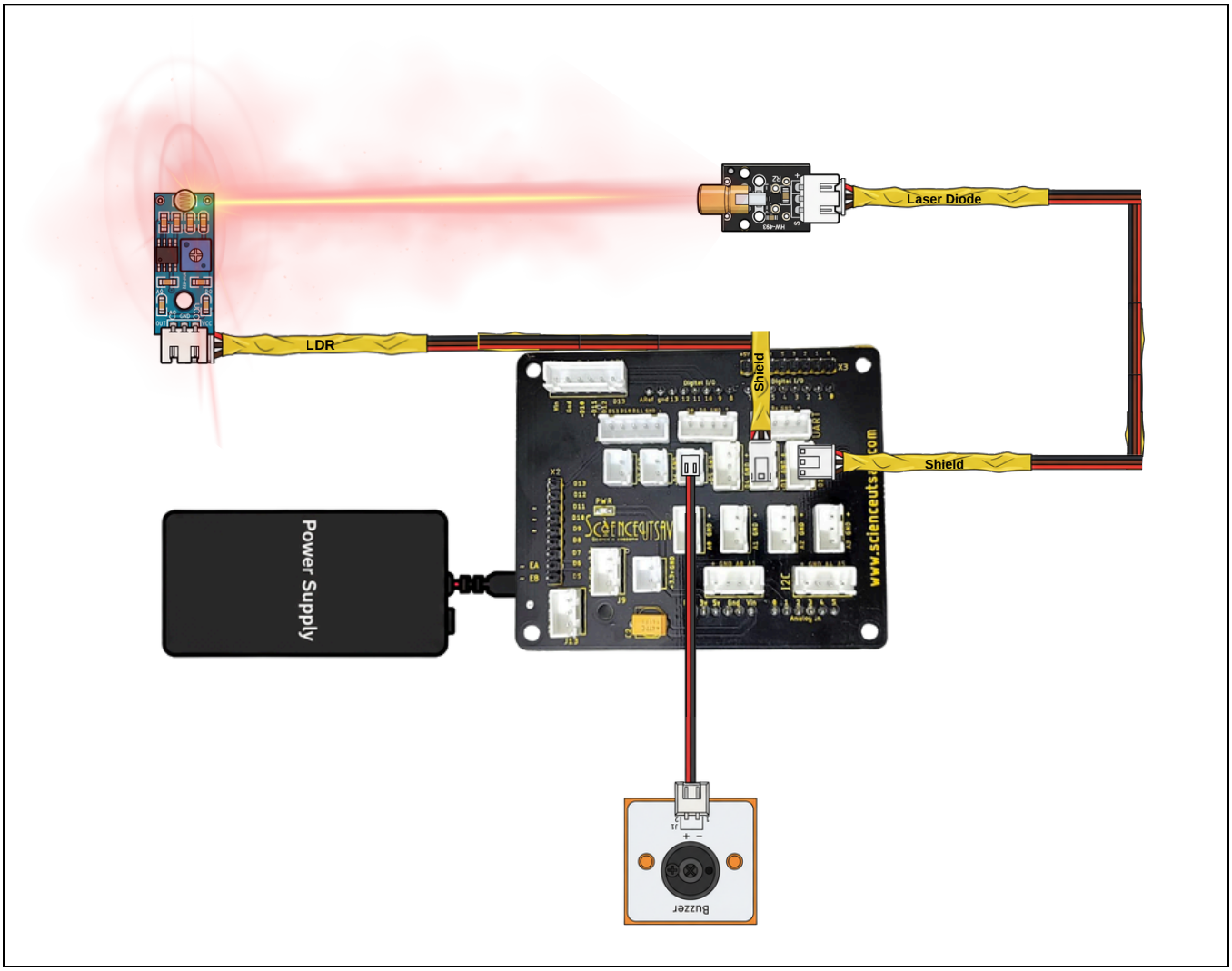


**Step 4 :** Fix it with the spacer screw from the bottom as shown below



# BEGINNER PROJECT

The circuit diagram of the project is given below




## CONNECTIONS:

1. LDR sensor-D3
2. Laser Diode-D2
3. Buzzer-D5

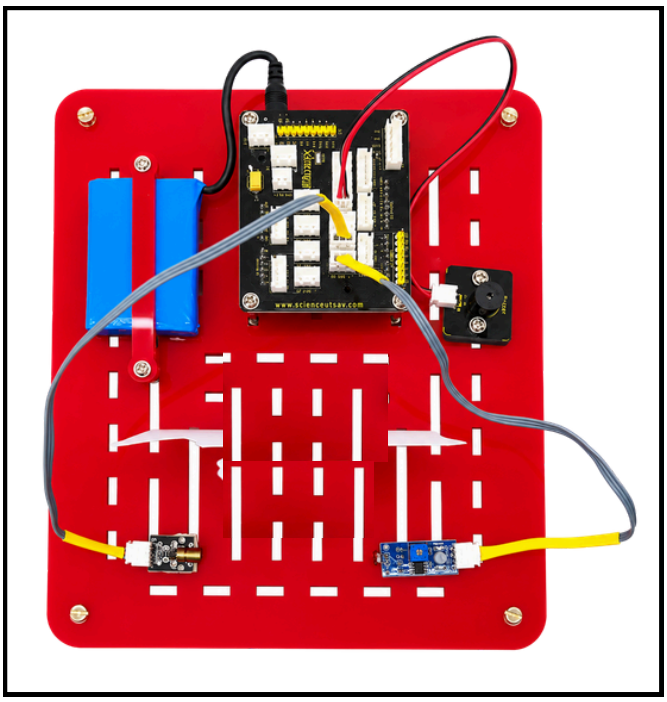
**Step 1-** Attach the stand to the L-clamp



**Step 2-** Attach the template to the stand as shown

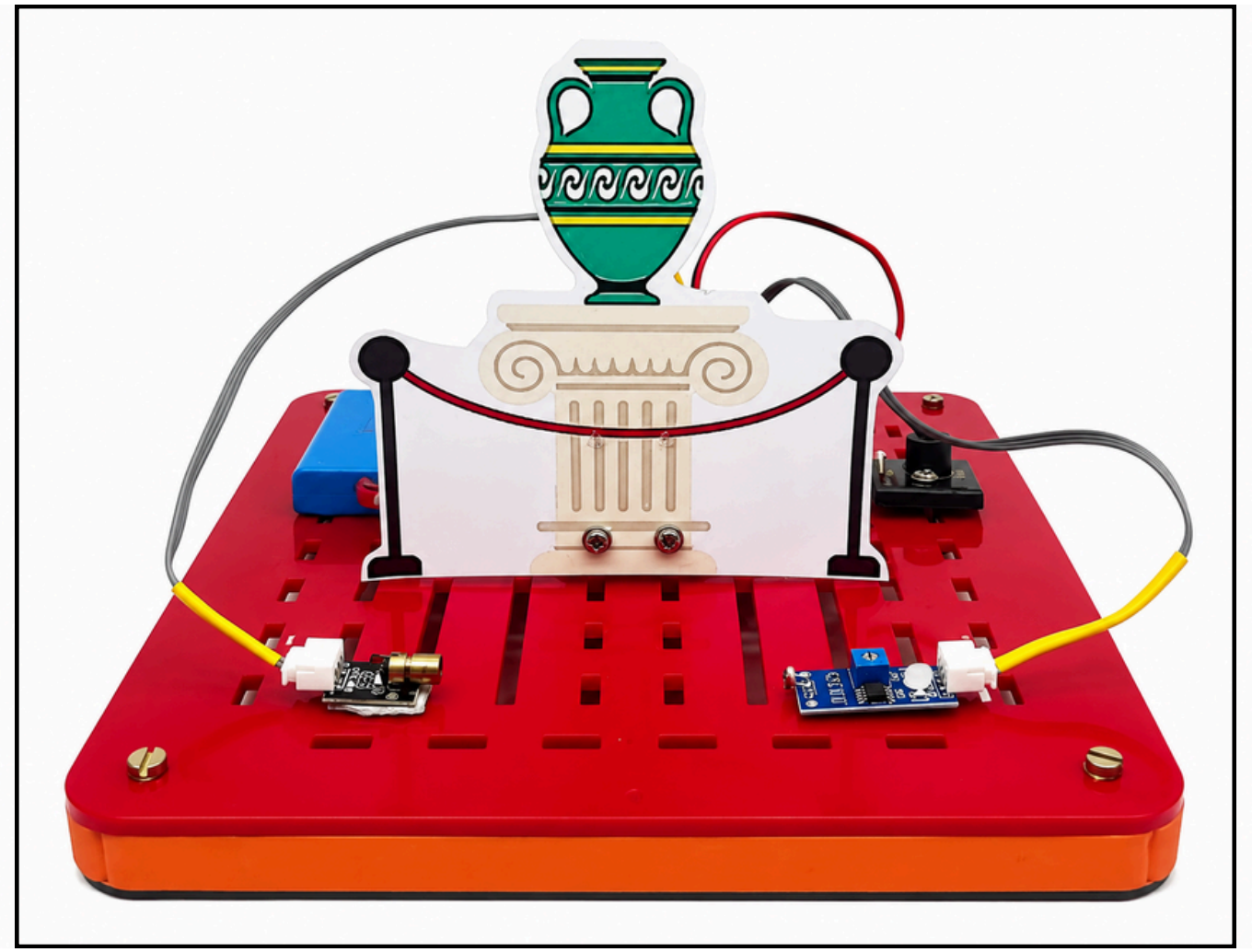


**Step 3-** Connect the buzzer, LDR and the laser diode to the board



**Remember-** The LDR and the laser diode must face each other.

## **Step 4-** Fix the template to the board



# Code

```
/*
Laser Security System
Laser  -> D2
LDR DO -> D3
Buzzer -> D5

Buzzer ON when laser beam is disturbed.
*/

const int laserPin = 2;
const int ldrPin = 3;
const int buzzerPin = 5;

// Most LDR digital modules give HIGH when light is received.
// If your buzzer works opposite, change this to LOW.
const int LASER_RECEIVED_STATE = HIGH;

void setup() {
  pinMode(laserPin, OUTPUT);
  pinMode(ldrPin, INPUT);
  pinMode(buzzerPin, OUTPUT);

  digitalWrite(laserPin, HIGH);   // Laser always ON
  digitalWrite(buzzerPin, LOW);   // Buzzer OFF at start

  Serial.begin(9600);
}

void loop() {
  int ldrState = digitalRead(ldrPin);

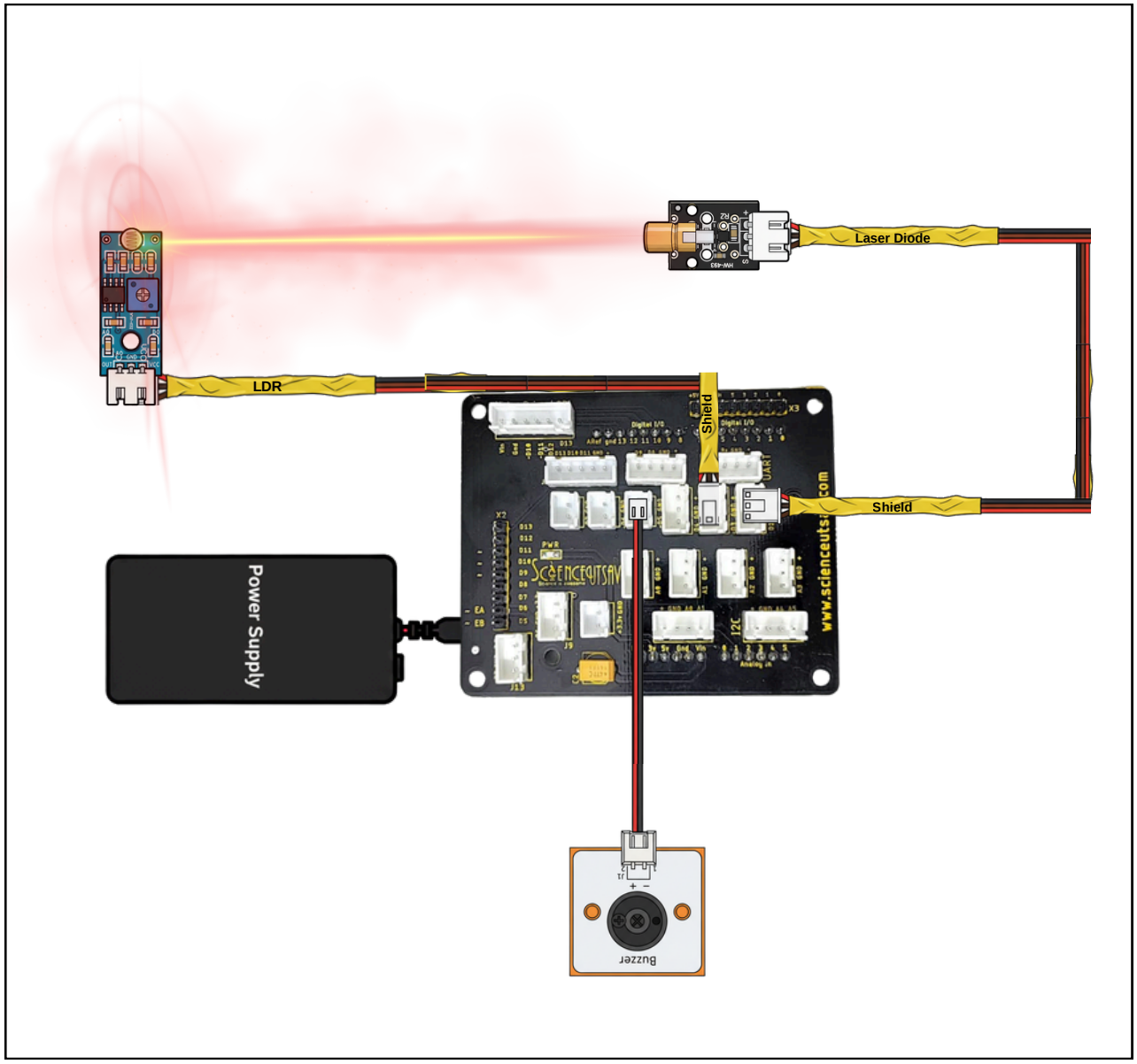
  Serial.print("LDR State: ");
  Serial.println(ldrState);

  // If laser is not falling on LDR, buzzer turns ON
  if (ldrState != LASER_RECEIVED_STATE) {
    digitalWrite(buzzerPin, HIGH);   // Buzzer ON
  } else {
    digitalWrite(buzzerPin, LOW);    // Buzzer OFF
  }

  delay(50);
}
```

# INTERMEDIATE PROJECT

The circuit diagram of the project is given below




## CONNECTIONS:

1. LDR sensor-D3
2. Laser Diode-D2
3. Buzzer-D5

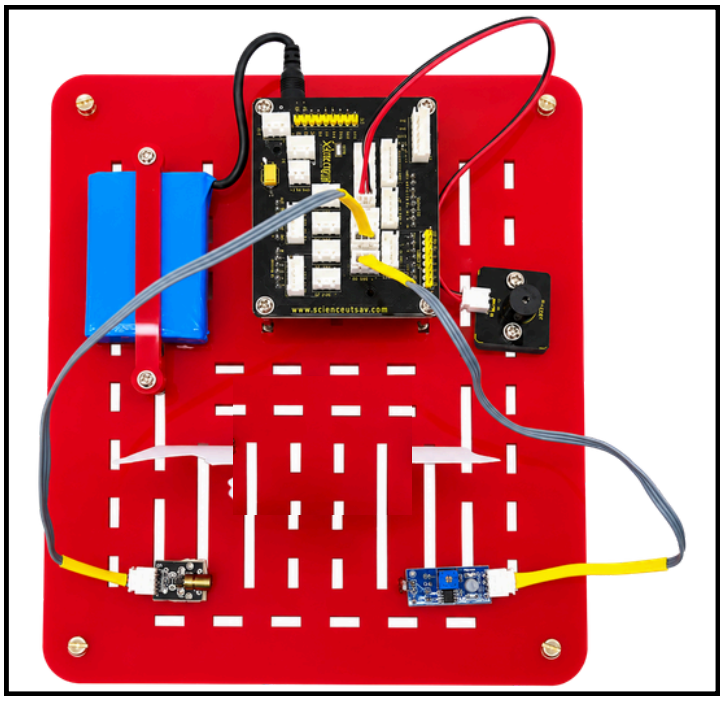
**Step 1-** Attach the stand to the L-clamp



**Step 2-** Attach the template to the stand as shown

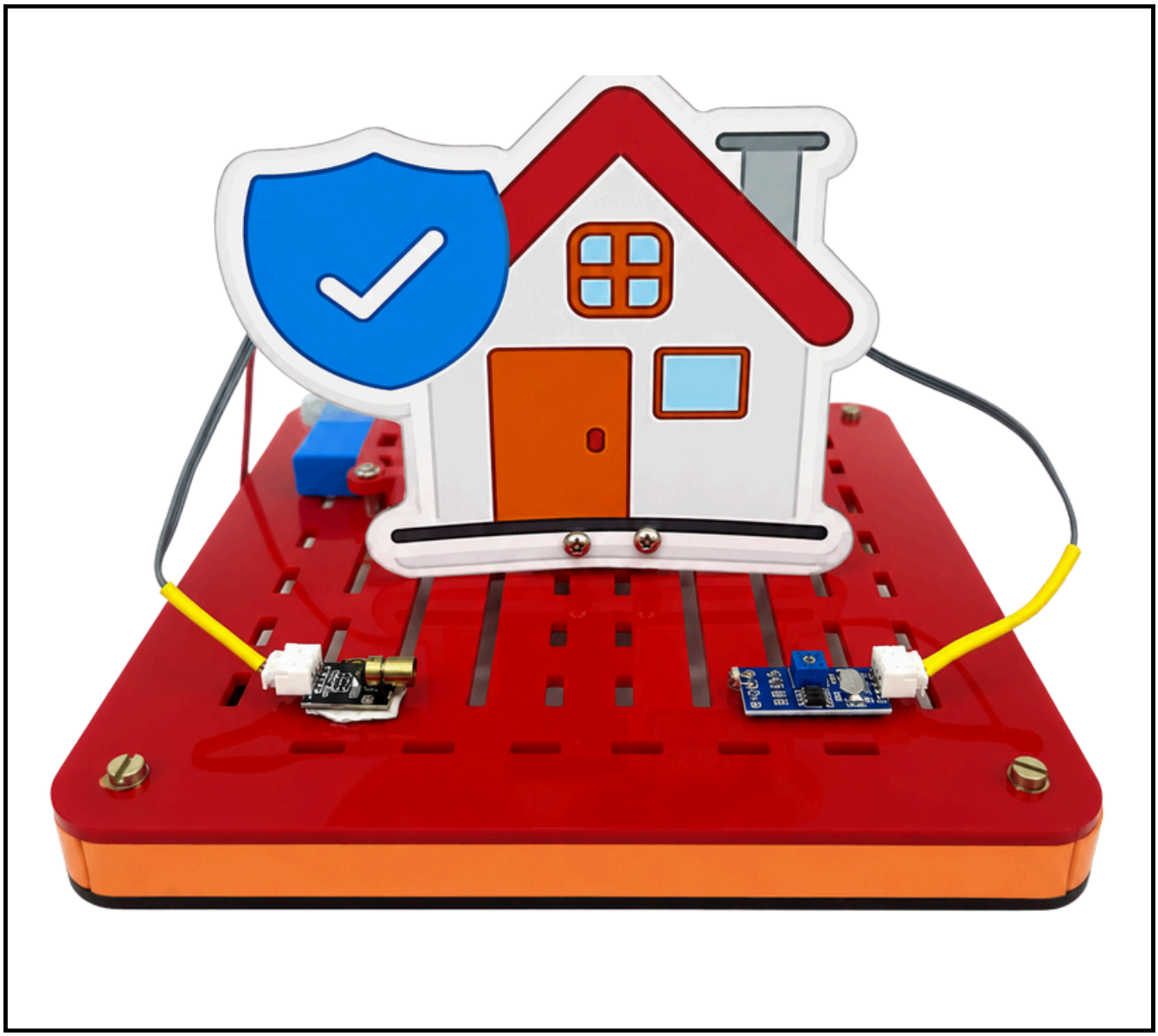


**Step 3-** Connect the buzzer, LDR and the laser diode to the board



**Remember-** The LDR and the laser diode must face each other.

## **Step 4-** Fix the template to the board



# Code

```
/*
Laser Security System
Laser  -> D2
LDR DO -> D3
Buzzer -> D5

Buzzer ON when laser beam is disturbed.
*/

const int laserPin = 2;
const int ldrPin = 3;
const int buzzerPin = 5;

// Most LDR digital modules give HIGH when light is received.
// If your buzzer works opposite, change this to LOW.
const int LASER_RECEIVED_STATE = HIGH;

void setup() {
  pinMode(laserPin, OUTPUT);
  pinMode(ldrPin, INPUT);
  pinMode(buzzerPin, OUTPUT);

  digitalWrite(laserPin, HIGH);   // Laser always ON
  digitalWrite(buzzerPin, LOW);   // Buzzer OFF at start

  Serial.begin(9600);
}

void loop() {
  int ldrState = digitalRead(ldrPin);

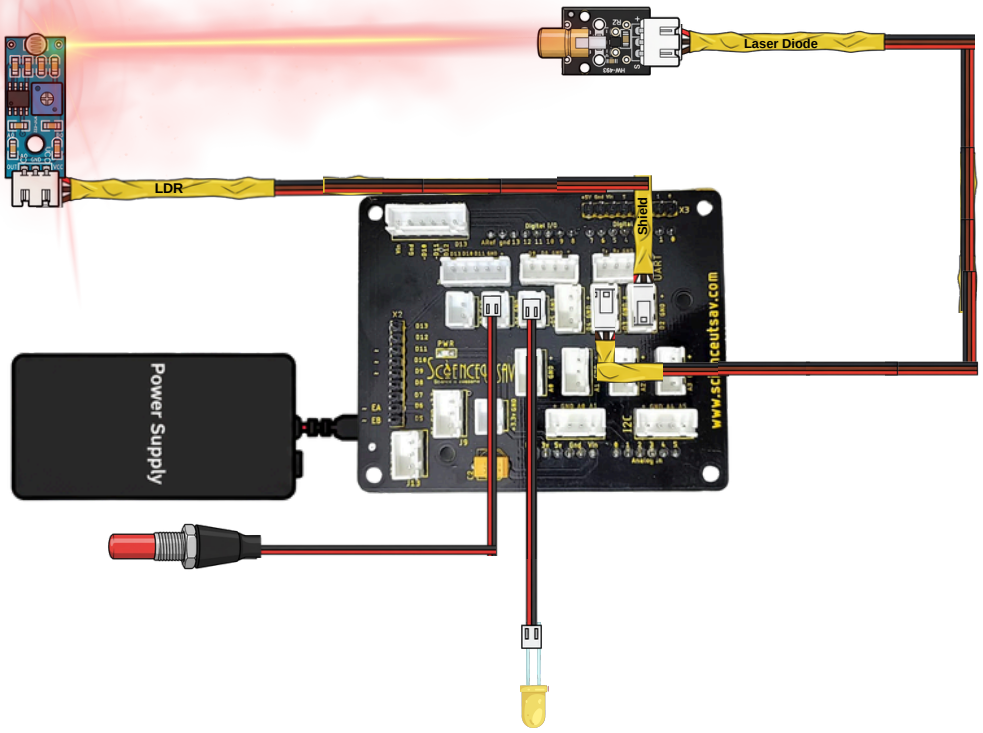
  Serial.print("LDR State: ");
  Serial.println(ldrState);

  // If laser is not falling on LDR, buzzer turns ON
  if (ldrState != LASER_RECEIVED_STATE) {
    digitalWrite(buzzerPin, HIGH);   // Buzzer ON
  } else {
    digitalWrite(buzzerPin, LOW);    // Buzzer OFF
  }

  delay(50);
}
```

# ADVANCE PROJECT

The circuit diagram of the project is given below



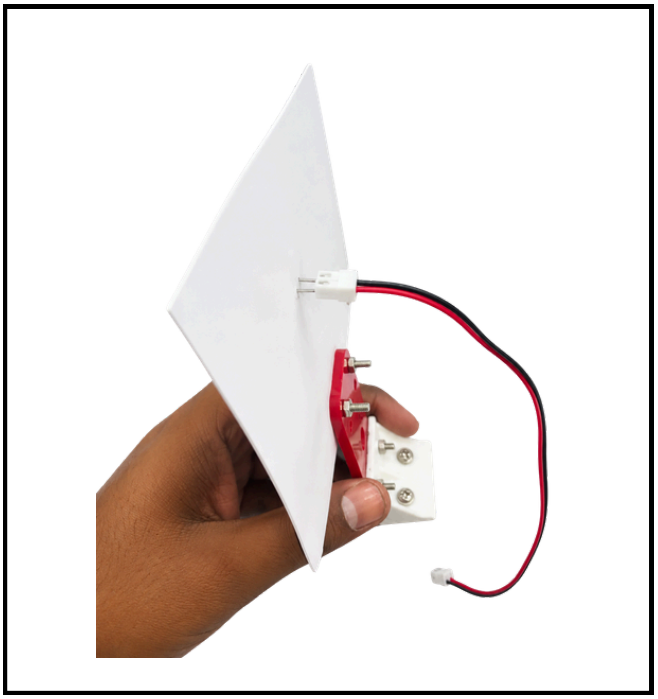

## CONNECTIONS:

1. LDR sensor-D3
2. Laser Diode-D2
3. LED-D5
4. Push button-D6

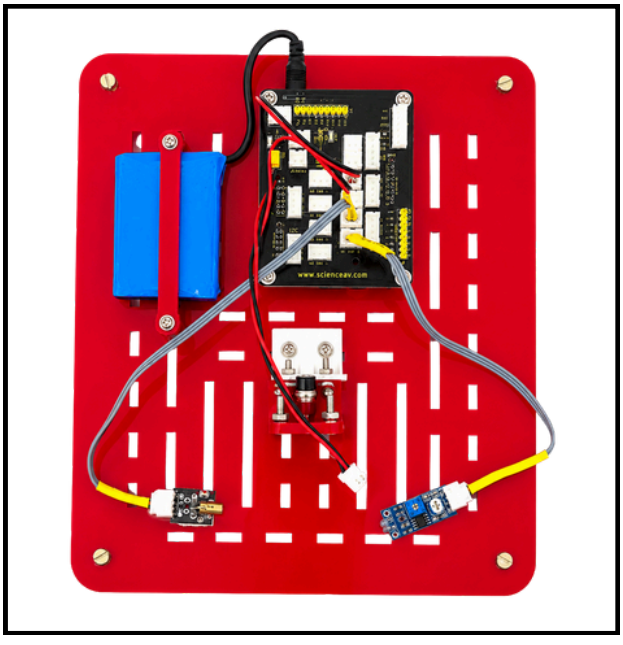
**Step1-** Attach the template to the stand as shown



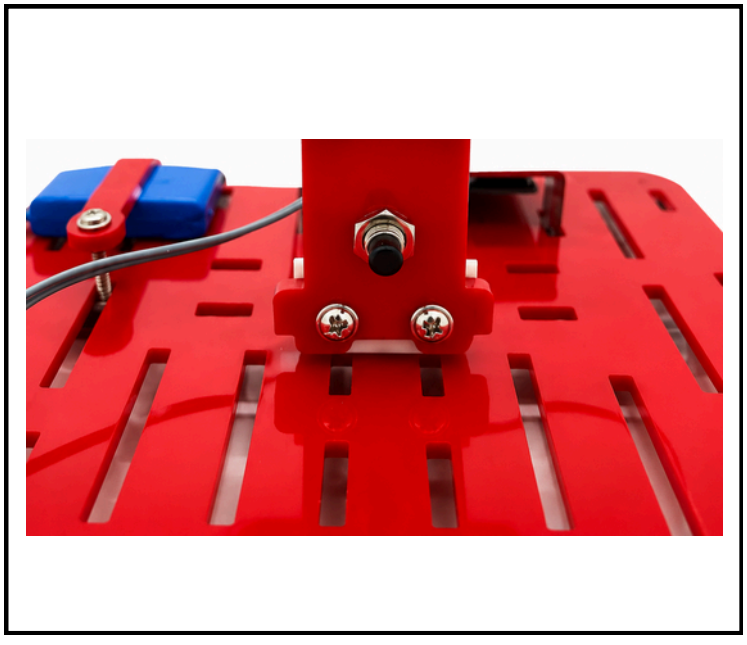
**Step 2-** Fix the LED to the template



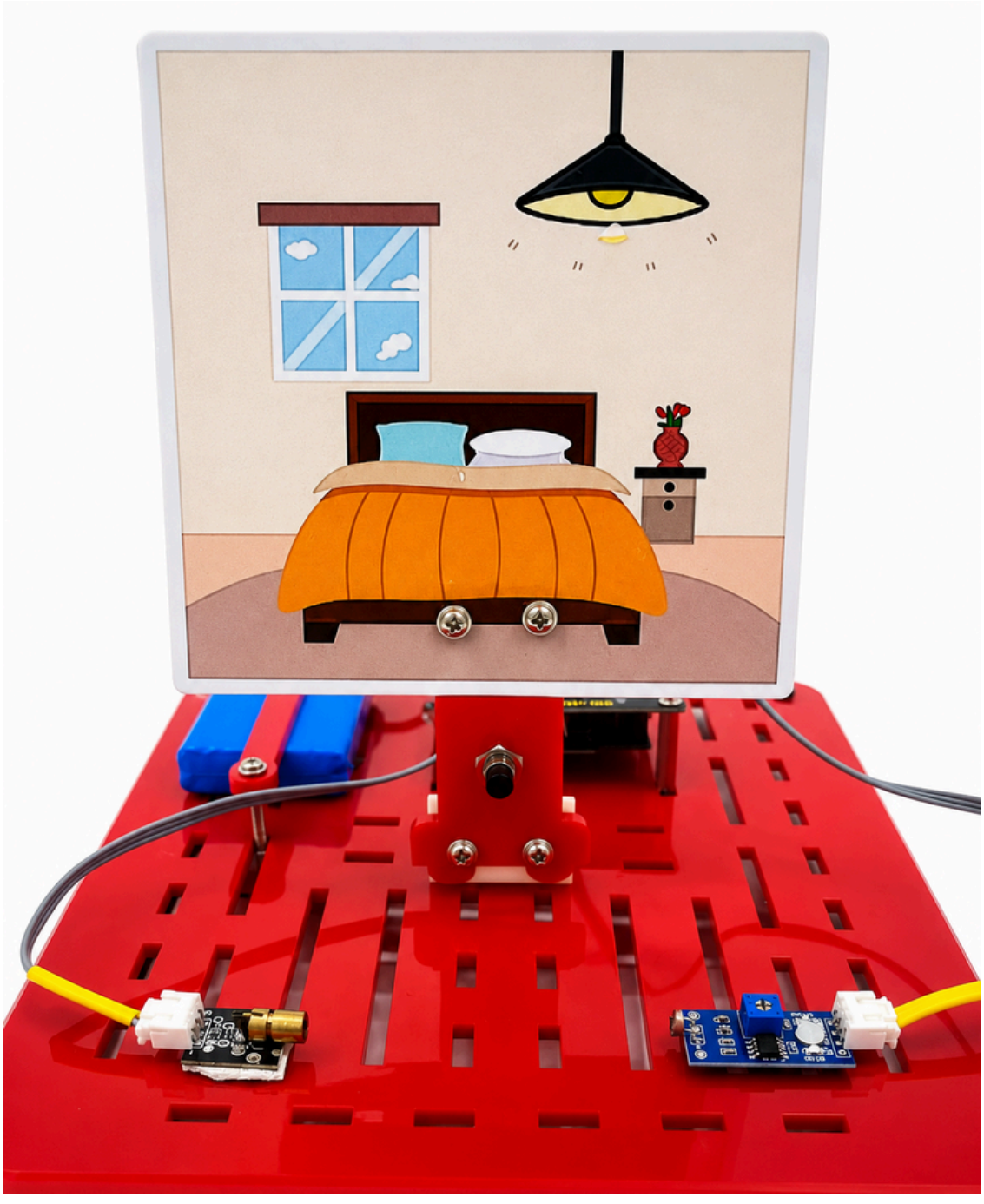
**Step 3-** Connect the LED, LDR and the laser diode to the board



**Step 4-** Fix the stand to the board and insert the push button as shown



**Step 5-** Upload the code annd observe



# Code

```
/*
ScienceUtsav Arduino Code - Session 6 to 10
Corrected for 3-pin DIGITAL LDR module.
LDR uses digitalRead(), not analogRead().
*/

const int ldrSensor = 2;
const int laser = 3;     // 3-pin laser SIGNAL pin
const int led = 5;
const int buzzer = 6;
const int BEAM_BROKEN_STATE = LOW; // If opposite, change LOW to
HIGH

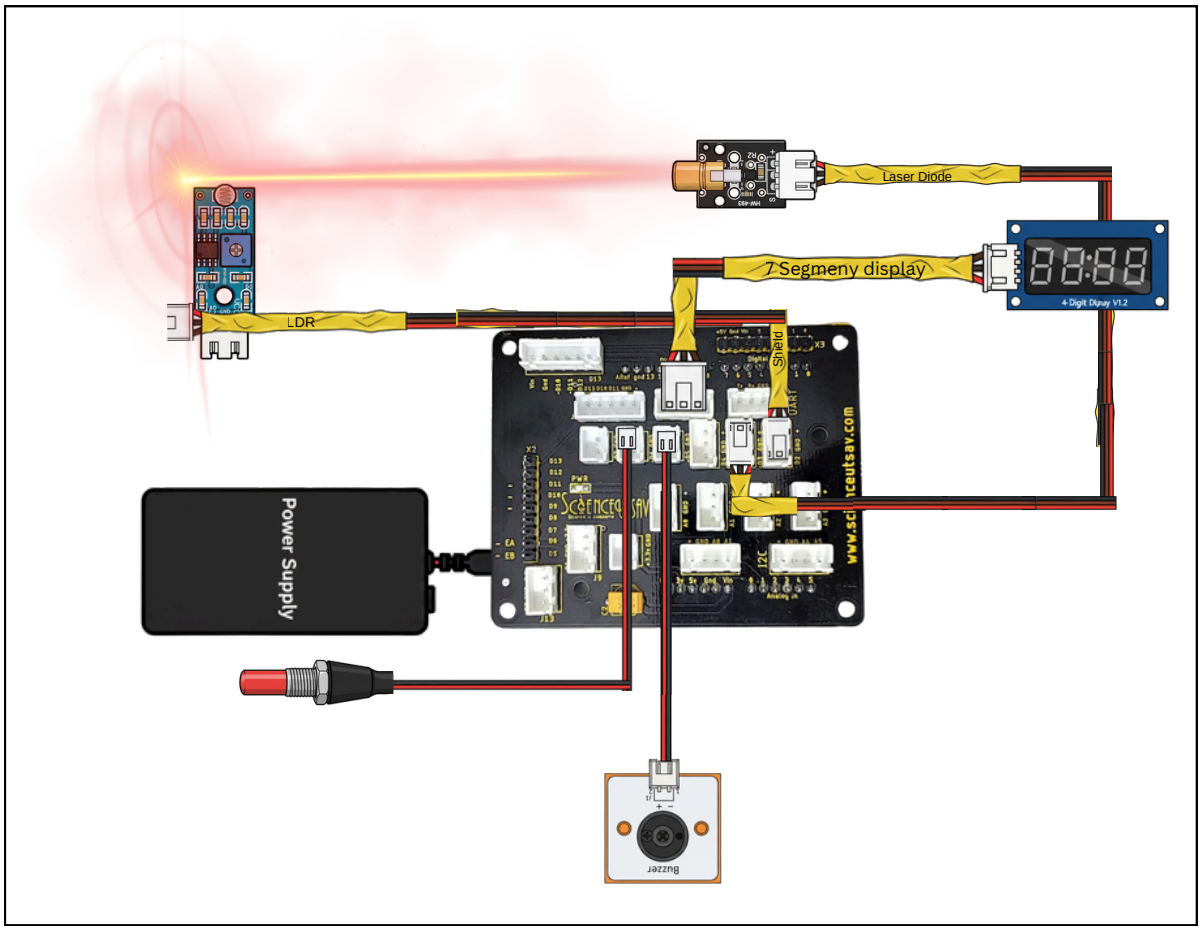
void setup() {
  pinMode(ldrSensor, INPUT);
  pinMode(laser, OUTPUT);
  pinMode(led, OUTPUT);
  pinMode(buzzer, OUTPUT);
  digitalWrite(laser, HIGH);
}

void loop() {
  bool beamBroken = digitalRead(ldrSensor) ==
BEAM_BROKEN_STATE;

  if (beamBroken) {
    digitalWrite(led, HIGH);
    tone(buzzer, 1200);
  } else {
    digitalWrite(led, LOW);
    noTone(buzzer);
  }
}
```

# EXPERT PROJECT

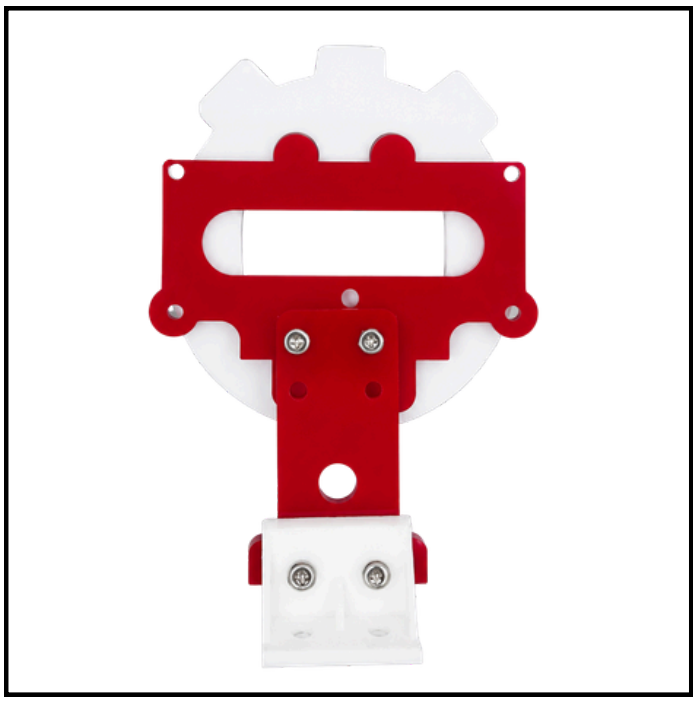
The circuit diagram of the project is given below



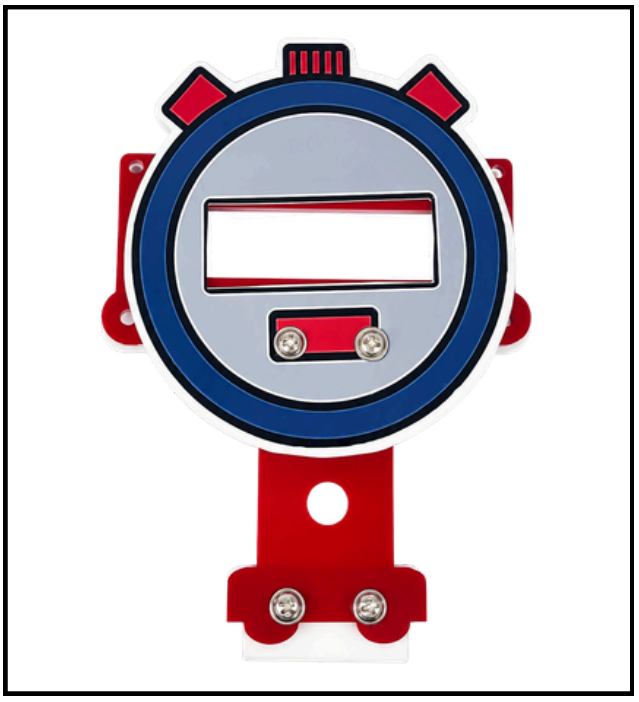

## CONNECTIONS:

1. LDR sensor-D2
2. Laser Diode-D3
3. Buzzer-D5
4. Push button-D6
5. 7 segment display-D8, D9

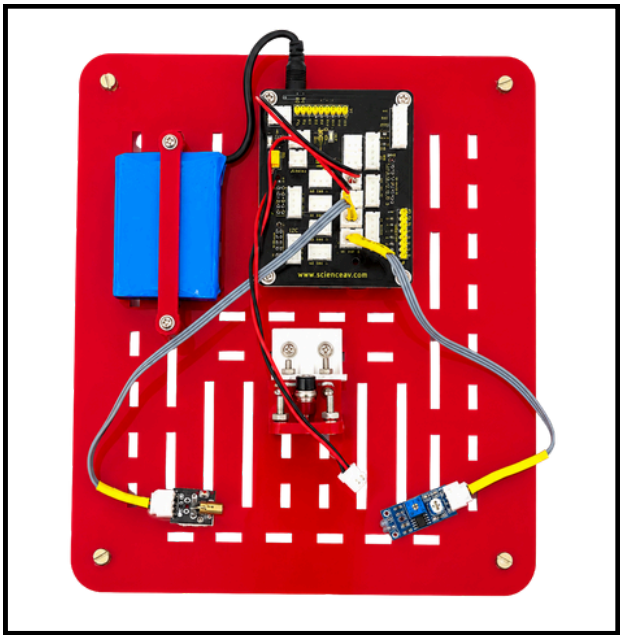
**Step 1-** Attach both the stands to each other as shown



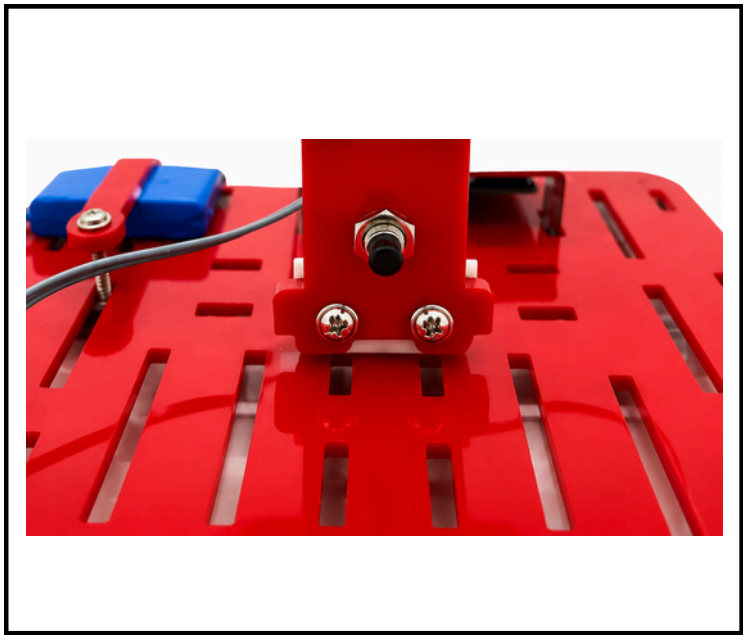
**Step 2-** Attach the template to the stand



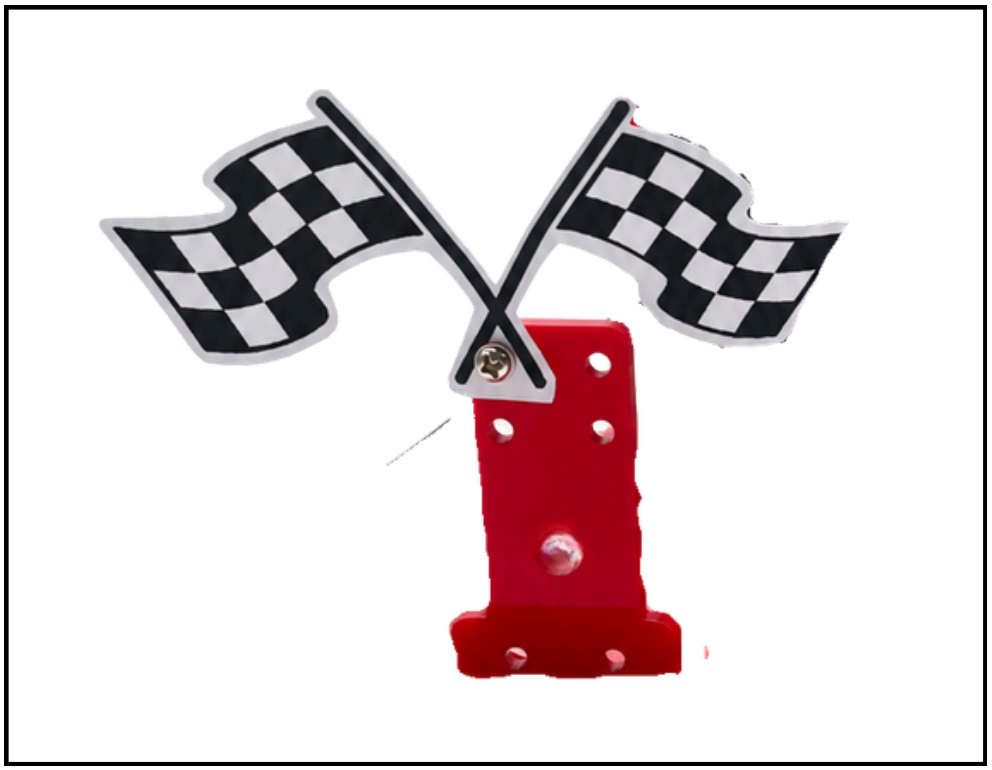
**Step 3-** Connect the LED, LDR and the laser diode to the board

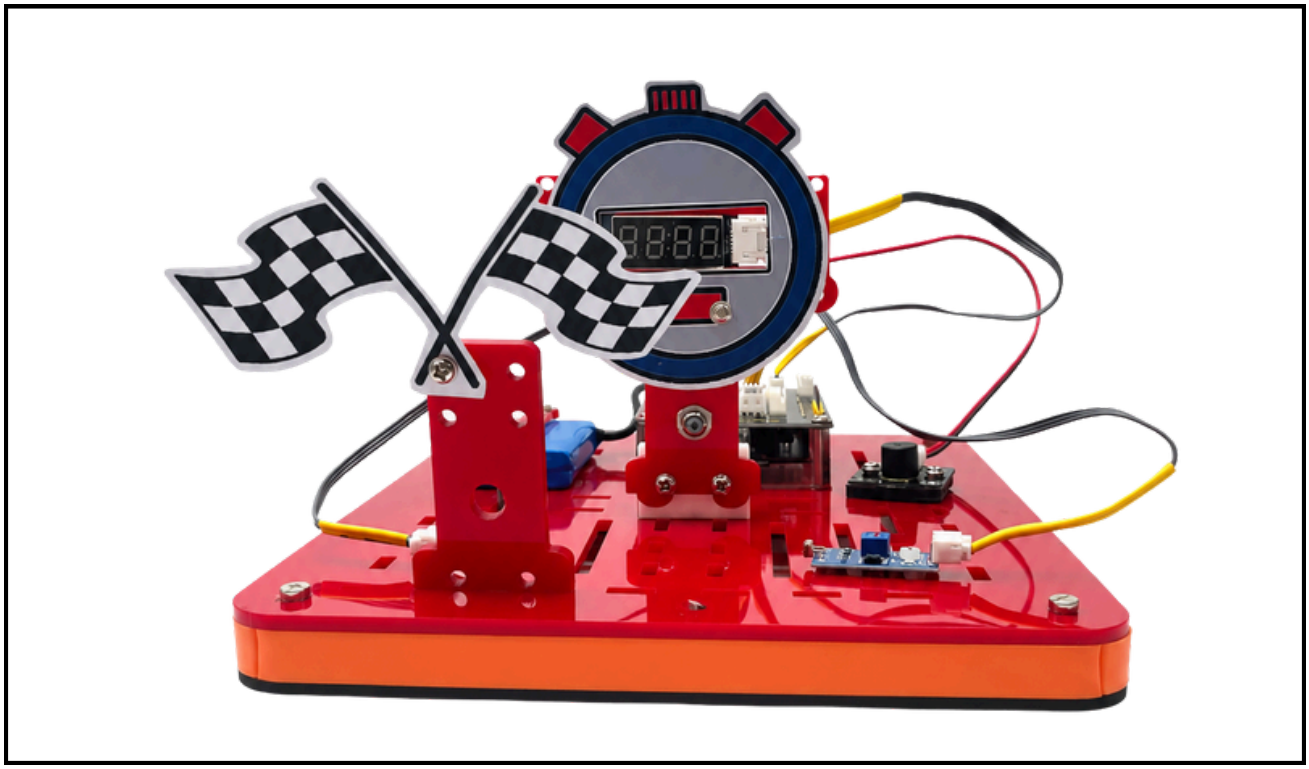


**Step 4-** Fix the stand to the board and insert the push button as shown



**Step 5 -** Fix the other template to the stand as shown





# Code

```
/*
Laser Race Timer Project

Laser  -> D2
LDR DO -> D3
Button -> D4
Buzzer -> D5

TM1637 CLK -> D8
TM1637 DIO -> D9

Display format:
12:45 means 12 seconds and 45 centiseconds
*/

const int laserPin = 2;
const int ldrPin = 3;
const int buttonPin = 6;
const int buzzerPin = 5;

const int CLK = 8;
const int DIO = 9;

TM1637Display display(CLK, DIO);

// Most LDR digital modules give HIGH when laser/light falls on it.
// If your project works opposite, change HIGH to LOW.
const int LASER_RECEIVED_STATE = HIGH;

enum RaceState {
  IDLE,
  RUNNING,
  FINISHED
};

RaceState raceState = IDLE;

unsigned long raceStartTime = 0;
unsigned long finalTime = 0;

bool lastButtonState = HIGH;
unsigned long lastButtonPressTime = 0;
const unsigned long debounceDelay = 250;
```

```
// Ignore LDR for first 500 ms after race starts
// This avoids false trigger at starting time
const unsigned long startIgnoreTime = 500;

void showTime(unsigned long elapsedTime) {
 unsigned int centiseconds = (elapsedTime / 10) % 100;
 unsigned int seconds = (elapsedTime / 1000) % 100;

 int displayValue = (seconds * 100) + centiseconds;

 // Shows SS:CC
 display.showNumberDecEx(displayValue, 0b01000000, true);
}

void winningSound() {
 tone(buzzerPin, 1000, 150);
 delay(200);
 tone(buzzerPin, 1500, 150);
 delay(200);
 tone(buzzerPin, 2000, 250);
 delay(300);
 tone(buzzerPin, 2500, 400);
 delay(450);
 noTone(buzzerPin);
}

bool buttonPressedOnce() {
 bool currentButtonState = digitalRead(buttonPin);

 if (lastButtonState == HIGH && currentButtonState == LOW) {
 if (millis() - lastButtonPressTime > debounceDelay) {
 lastButtonPressTime = millis();
 lastButtonState = currentButtonState;
 return true;
 }
 }

 lastButtonState = currentButtonState;
 return false;
}

bool laserBeamDisturbed() {
 int ldrState = digitalRead(ldrPin);
```

```
// If LDR is not receiving laser, beam is disturbed
 return ldrState != LASER_RECEIVED_STATE;
}

void startRace() {
 raceState = RUNNING;
 raceStartTime = millis();
 finalTime = 0;

 digitalWrite(laserPin, HIGH);
 noTone(buzzerPin);

 display.showNumberDecEx(0, 0b01000000, true);

 Serial.println("Race Started");
}

void finishRace() {
 raceState = FINISHED;
 finalTime = millis() - raceStartTime;

 digitalWrite(laserPin, LOW);
 showTime(finalTime);

 Serial.print("Winner Time: ");
 Serial.print(finalTime / 1000.0);
 Serial.println(" seconds");

 winningSound();
}



void setup() {
 pinMode(laserPin, OUTPUT);
 pinMode(ldrPin, INPUT);
 pinMode(buttonPin, INPUT_PULLUP);
 pinMode(buzzerPin, OUTPUT);

 digitalWrite(laserPin, LOW);
 digitalWrite(buzzerPin, LOW);

 display.setBrightness(7);
 display.showNumberDecEx(0, 0b01000000, true);
```

```
Serial.begin(9600);
 Serial.println("Laser Race Timer Ready");
}

void loop() {
 if (buttonPressedOnce()) {
 startRace();
 }

 if (raceState == RUNNING) {
 unsigned long elapsedTime = millis() - raceStartTime;

 showTime(elapsedTime);

 // Check LDR only after small starting delay
 if (elapsedTime > startIgnoreTime) {
 if (laserBeamDisturbed()) {
 finishRace();
 }
 }
 }

 if (raceState == FINISHED) {
 // Timer stays frozen here.
 // Next button press will start next race.
 }
}
```